Задание №27 - DBSCAN

Задание 1

1. Загрузка данных: на первом этапе открываем файл, при этом первую строку пропускаем. Все точки данных из файла считываем построчно и преобразуем в вещественный тип чисел (float), после чего сохраняем точки в виде списка points.

2. Функция centroid для вычисления центроида: принимает на вход список точек, принадлежащих одному кластеру. Для каждой точки в кластере вычисляется сумма её расстояний до всех остальных точек внутри этого же кластера с использованием math.dist(). Точка, имеющая минимальную сумму расстояний до других точек, выбирается в качестве центроида.

3. Инициализация кластеров:
INSANEK.RU
Создаём пустой список clusters для хранения найденных кластеров.
Определяем пороговое расстояние epsilon = 0.5.

4. Формирование кластеров:

Пока в списке points остаются нераспределённые точки, выбирается первая точка и создаётся новый кластер.
Точка добавляется в новый кластер, затем она удаляется из списка points.
Для каждой точки в новом кластере проверяется расстояние до остальных точек.
Если точка находится в пределах epsilon, она добавляется в текущий кластер, а затем удаляется из points.

5. Визуализация кластеров:

Используем библиотеку turtle, где каждая точка выводится в определённом цвете.
Координаты точек масштабируем с коэффициентом k = 40.
Для каждой точки вызывается setpos(), а затем dot(5, color[i]), чтобы отобразить точки разными цветами.

7. Вычисление центроидов:

Для каждого кластера вычисляем его центроид с помощью функции centroid().
Итоговые координаты центроидов умножаются на 10000 и выводятся в консоль.

Полный код программы для файла A:

```
import math

from turtle import *

from math import dist
```

```
def visual(clusters):
    k = 40
    penup(), tracer(0)
    colors = ['green', 'blue', 'brown', 'red', 'yellow', 'pink', 'black']
    for i in range(len(clusters)):
        for x, y in clusters[i]:
            setpos(x*k, y*k)
            dot(5, colors[i])
    done()
```

INSANEK.RU

```
def centroid(cluster):
    c = []
    for x1, y1 in cluster:
        c += [[sum(math.dist((x1, y1), (x2, y2)) for x2, y2 in cluster), (x1, y1)]]
    return min(c)[1]


f = open('27dbscan/27_1/27A.txt')
f.readline()
points = [list(map(float, s.replace(",", ".").split())) for s in f]
clusters, epsilon = [], 0.5
while points:
    clusters.append([points[0]])
    del points[0]
    for p1 in clusters[-1]:
        for p2 in points[:]:
            if math.dist(p1, p2) < epsilon:
```

```
            clusters[-1].append(p2)
            points.remove(p2)
    visual(clusters)
    Px = sum([centroid(clusters[i])[0] for i in range(len(clusters))]) / len(clusters)
    Py = sum([centroid(clusters[i])[1] for i in range(len(clusters))]) / len(clusters)
    print(abs(Px * 10000) // 1, abs(Py * 10000) // 1)
```

Полный код программы для файла Б:

INSANEK.RU

```
import math
from turtle import *
from math import dist
def visual(clusters):
    k = 40
    penup(), tracer(0)
    colors = ['green', 'blue', 'brown', 'red', 'yellow', 'pink', 'black']
    for i in range(len(clusters)):
        for x, y in clusters[i]:
            setpos(x*k, y*k)
            dot(5, colors[i])
    done()

def centroid(cluster):
    c = []
    for x1, y1 in cluster:
        c += [[sum(math.dist((x1, y1), (x2, y2)) for x2, y2 in cluster), (x1, y1)]]
    return min(c)[1]
```

```
f = open('27dbscan/27_1/27B.txt')
f.readline()
points = [list(map(float, s.replace(",", ".").split())) for s in f]
clusters, epsilon = [], 0.5
while points:
    clusters.append([points[0]])
    del points[0]
    for p1 in clusters[-1]:
        for p2 in points[:]:
            if math.dist(p1, p2) < epsilon:
                clusters[-1].append(p2)
                points.remove(p2)
visual(clusters)
Px = sum([centroid(clusters[i])[0] for i in range(len(clusters))]) / len(clusters)
Py = sum([centroid(clusters[i])[1] for i in range(len(clusters))]) / len(clusters)
print(abs(Px * 10000) // 1, abs(Py * 10000) // 1)
```

INSANEK.RU

Получаем ответ:

8445, 14977

2641, 2714

Задание 2

1. Загрузка данных: на первом этапе открываем файл, при этом первую строку пропускаем. Все точки данных из файла считываем построчно и преобразуем в вещественный тип чисел (float), после чего сохраняем точки в виде списка points.

2. Функция centroid для вычисления центроида: принимает на вход список точек, принадлежащих одному кластеру. Для каждой точки в кластере вычисляется сумма её расстояний до всех остальных точек внутри этого же кластера с использованием math.dist(). Точка, имеющая минимальную сумму расстояний до других точек, выбирается в качестве центроида.

3. Инициализация кластеров:
Создаём пустой список clusters для хранения найденных кластеров.
Определяем пороговое расстояние epsilon = 0.25.

INSANER.RU

4. Формирование кластеров:

Пока в списке points остаются нераспределённые точки, выбирается первая точка и создаётся новый кластер.
Точка добавляется в новый кластер, затем она удаляется из списка points.
Для каждой точки в новом кластере проверяется расстояние до остальных точек.
Если точка находится в пределах epsilon, она добавляется в текущий кластер, а затем удаляется из points.

5. Визуализация кластеров:

Используем библиотеку turtle, где каждая точка выводится в определённом цвете.
Координаты точек масштабируем с коэффициентом k = 40.
Для каждой точки вызывается setpos(), а затем dot(5, color[i]), чтобы отобразить точки разными цветами.

7. Вычисление итоговых значений:

Для каждого кластера вычисляем его центроид с помощью функции centroid(), а также его площадь деля количество точек в кластере на площадь кластера ($H \cdot W = 4 \cdot 4 = 16$).
Итоговые координаты центроидов умножаются на 10000 и выводятся в консоль, также P_s умножаем на 1000 и целую часть от произведения выводим на экран.

Полный код программы для файла A:

```
import math

from turtle import *
```

```
from math import dist
def visual(clusters):
    k = 40
    penup(), tracer(0)
    colors = ['green', 'blue', 'brown', 'red', 'yellow', 'pink', 'black']
    for i in range(len(clusters)):
        for x, y in clusters[i]:
            setpos(x*k, y*k)
            dot(5, colors[i])
    done()


def centroid(cluster):
    c = []
    for x1, y1 in cluster:
        c += [[sum(math.dist((x1, y1), (x2, y2)) for x2, y2 in cluster), (x1, y1)]]
    return min(c)[1]


f = open('27dbscan/27_2/27A.txt')
f.readline()
points = [list(map(float, s.replace(",", ".").split())) for s in f]
clusters, epsilon = [], 0.25
while points:
    clusters.append([points[0]])
    del points[0]
    for p1 in clusters[-1]:
        for p2 in points[:]:
```

INSANEK.RU

```
        if math.dist(p1, p2) < epsilon:
            clusters[-1].append(p2)
            points.remove(p2)
    if len(clusters[-1]) <= 10:
        del clusters[-1]

best_centroids = [[] for i in range (len(clusters))]
for i in range (len(clusters)):
    min_dist = 10**10
    for x1, y1 in clusters[i]:
        dist = 0
        for x2, y2 in clusters[i]:
            euclidian_dist = math.dist([x1, y1], [x2, y2])
            dist += euclidian_dist
        if dist < min_dist:
            min_dist = dist
            best_centroids[i] = [x1, y1]
print(best_centroids)
visual(clusters)
P_x = sum([x for x, y in best_centroids]) / len(clusters)
P_y = sum([y for x, y in best_centroids]) / len(clusters)
P_s = sum([len(cluster) / (4 * 4) for cluster in clusters])
print(int(abs(P_x + P_y) * 10000), int(abs(P_s * 1000)))
```

INSANEK.RU

Полный код программы для файла Б:

```
import math
from turtle import *
```

```
from math import dist


def visual(clusters):
    k = 40
    penup(), tracer(0)
    colors = ['green', 'blue', 'brown', 'red', 'yellow', 'pink', 'black']
    for i in range(len(clusters)):
        for x, y in clusters[i]:
            setpos(x*k, y*k)
            dot(5, colors[i])
    done()


def centroid(cluster):
    c = []
    for x1, y1 in cluster:
        c += [[sum(math.dist((x1, y1), (x2, y2)) for x2, y2 in cluster), (x1, y1)]]
    return min(c)[1]


f = open('27dbscan/27_2/27B.txt')
f.readline()
points = [list(map(float, s.replace(",", ".").split())) for s in f]
clusters, epsilon = [], 0.25


while points:
    clusters.append([points[0]])
    del points[0]
```

INSANEK.RU

```
    for p1 in clusters[-1]:
        for p2 in points[:]:
            if math.dist(p1, p2) < epsilon:
                clusters[-1].append(p2)
                points.remove(p2)
    if len(clusters[-1]) <= 10:
        del clusters[-1]


best_centroids = [[] for i in range (len(clusters))]
for i in range (len(clusters)):
    min_dist = 10**10
    for x1, y1 in clusters[i]:
        dist = 0
        for x2, y2 in clusters[i]:
            euclidian_dist = math.dist([x1, y1], [x2, y2])
            dist += euclidian_dist
        if dist < min_dist:
            min_dist = dist
            best_centroids[i] = [x1, y1]


print(best_centroids)
visual(clusters)


P_x = sum([x for x, y in best_centroids]) / len(clusters)
P_y = sum([y for x, y in best_centroids]) / len(clusters)
P_s = sum([len(cluster) / (4 * 4) for cluster in clusters])
```

INSANEK.RU

```
print(int(abs(P_x + P_y) * 10000), int(abs(P_s * 1000)))
```

Получаем ответ:

28945, 61375

8665, 656875

INSANEK.RU